DNSSEC: A Study into Manual Signing and the Validation of Signatures

by

Steven Dormady

Advised by

Dr. Mohamed Aboutabl

An Independent Study Project

Department of Computer Science

James Madison University

Department of Computer Science

April 2026

Contents

Abstract **iv**

1 Introduction - Add this to Abstract? **1**

1.1 Motivation 1

1.2 Problem Statement 1

1.3 Contributions 2

1.4 VM Research 2

1.5 VM Network 3

1.6 Bind9 3

2 Introduction to the Domain Naming System **4**

2.1 DNS Background 4

2.1.1 DNS Resource Records 5

2.2 The DNS Environment 8

2.2.1 Environment Zones 9

2.3 DNS Vulnerabilities 11

2.3.1 DNS Spoofing/Cache Poisoning 11

2.3.2 DNS Hijacking 13

2.4 DNS Attacks 14

3 Introduction to DNS Security Extension **16**

3.1 DNSSEC Background 16

3.2 New DNS Resource Records 17
3.2.1 Resource Record Set 18
3.2.2 Resource Record Signature 19
3.2.3 DNSKEYs In General 21
3.2.4 Delegation Signer 22
3.3 Chain Of Trust 23
3.4 Key Rollovers 24
3.5 DNSSEC Options 24
3.5.1 DNSSEC Policy 24
3.5.2 Signing 25
3.6 DNSSEC In The Environment 25
3.7 Simulated Attacks with DNSSEC Enabled 25
3.7.1 Corrupting DNS Data 27

4 Algorithms and Data Structures For DNSSEC 29
4.1 An Introduction to DNSSEC Signing 29
4.1.1 Hashing 29
4.1.2 Signing 30
4.1.3 Key Conversion 30
4.2 RRSIG Construction 30
4.3 OpenSSL 31

5 Programming the Signature and Validation of Several DNSSEC Resource Records 32
5.1 Main Functionality 32
5.2 Converting Base64 Data 33
5.3 Hashing the Data 33
5.4 Signing the Data 34

5.5 Verifying the Signature 34
5.6 Generating and Validating an A RRSIG Record 35
5.7 Generating and Validating an NS RRSIG Record 35
5.8 Generating and Validating an DNSKEY RRSIG Record 35

A Appendix **39**

A.1 Procedure Manual 39

Abstract

TBD

Introduction - Add this to Abstract?

1.1 Motivation

DNS by itself is insecure and vulnerable to attack. With DNSSEC, these vulnerabilities are contained and mitigated.

My personal motivation for this study was a hand on security project, advised by Dr. Aboutabl, who has become one of my personal favorite professors here at JMU. His Information Security class helped further my interest for security, and when he mentioned this study, I knew I had to seize the opportunity.

1.2 Problem Statement

The problem we are trying to solve here is a dive into DNS Security Extension, or DNSSEC, and how it changes DNS. Using a testbed network of virtual machines on a virtual network, I was able to use Bind9 to be able to set up the DNS environment, then convert it to use DNSSEC. In this study of DNSSEC, I will be looking at the algorithms and data structures used in DNSSEC, and how they are used to secure DNS. I will also be looking at the performance of DNSSEC, and how it compares to DNS.

1.3 Contributions

I contributed the research and implementation of the DNS testbed environment, the algorithms and data structures used in DNSSEC, and the performance analysis of DNSSEC. I also contributed to the writing of this thesis, and the presentation of the results. I also manually wrote the code to generate signatures and validate them, which is a good way to test the validation process, because I can control the data that is being signed and verified. Not only that, but I am also creating the signature myself to prove that the received signature is valid as well.

Dr. Mohamed Aboutabl, my advisor, contributed in providing inspiration when I was stuck, and providing guidance on how to approach the project. He also provided feedback on the writing of this thesis, and the presentation of the results.

1.4 VM Research

I have never dealt directly with Virtual Machines before, and building a network of them was something I had never really thought about. However, the thought excited me and it seemed like a great opportunity to learn a new skill. Setting up the first one was a bit of a learning curve, but after setting up the first one to the specifications I wanted, I was able to clone the rest as needed.

The software I used to set up the testbed network for my DNSSEC program was VMWare Workstation. This software made it very easy to set up the virtual machines and network. Having a built in option to create a virtual network was very nice. I was also able to connect to the internet and download the necessary tools I needed, like Bind9, VMWare Tools, Wireshark, gcc and more. I also chose to use git for version control and being able to edit the code on multiple devices if needed.

1.5 VM Network

On VM Workstation, it is very easy and simple to set up a Virtual Network. Under the ”Edit” tab in the menu, I was able to create a new vmnet and customize it to my needs. I made a custom network, isolated from the outside networks. The IP I used was 192.168.107.X, with a subnet mask of 255.255.255.0. The 192.168 is an IP reserved for local networks, so this was perfect for me to to use. There was an option to enable or disable DHCP service. Enabling it meant that you could let DCHP assign addresses to machines on the network. For me, I wanted to statically and manually assign IPs without and DHCP intervention, so I deselected this option to have more control in my environment.

1.6 Bind9

I installed Bind9 on all of the VMs in the Virtual Network. The version I installed was 9.18.39. I decided on Bind9 because it was a good option to be used on Ubuntu machines and gave me the control I wanted.

Introduction to the Domain Naming System

2.1 DNS Background

Firstly, I think a bit of background about DNS helps to understand why it became what it is today. DNS started as ARPANET, which was very inefficient and used a centralized hosts file to map the domain names to IP addresses. When a change was made, it had to be redistributed to all of the systems. With number of users and systems needed increasing, this became very hard to maintain, which led to DNS today.[1]

In my research about DNS, I learned a lot about what happens behind the scenes with DNS Servers. Cloudflare called it, ”DNS is the phone book of the internet.”, and I think this name is very fitting.[2] Like a phonebook, you look up a name (domain name) and get someone’s phone number (IP address). All network systems operate with network addresses, such as IPv4 and IPv6. More or less, DNS translates domain names, like www.example.com, to IP addresses, like 134.126.126.99 (jmu.edu), so browsers can load internet resources.

The DNS naming structure is organized hierarchically in a tree-like format. Starting at the root of the tree, there is the Root Domain, which is where real world DNS begins. It delegates authority to the Top-Level Domains, or TLDs, which delegate further to Second-Level Domains, or SLDs, and continue onward, spreading out like a tree. Below in figure 2.1 is a diagram demonstrating this hierarchy.[1]

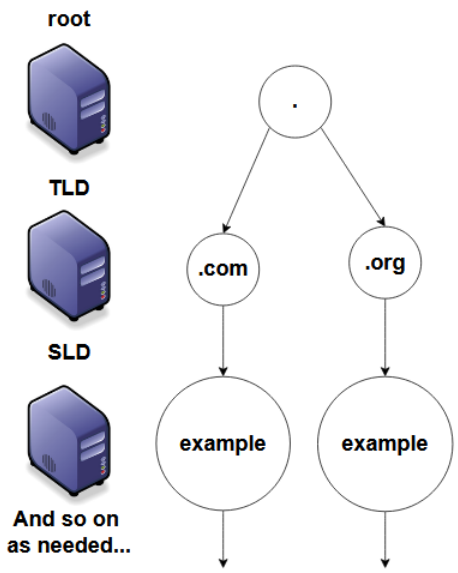

Figure 2.1: Root Delegation Hierarchy

A domain is the label of a node in the tree. A dot in a domain name means that it must traverse to another layer to get more information. Like in the above example, we start with ”example”, being at the Second-Level domain, then traverses to ”.com” at the Top-Level domain, then finally the root, ”.”, server. Put it all together and you get example.com., with the end period making the a Fully Qualified Domain Name, or FDQN. This period is usually hidden behind the scenes, and as users we do not use it. Root servers is the beginning of all DNS name resolutions, which makes the name root make sense. Like a tree, the DNS tree needs it’s roots to work. However, it helps to see the root domain being reached in this instance. There are 13 root servers due to historical reasons of how many addresses could be put into a 512 byte UDP message.[1]

2.1.1 DNS Resource Records

These DNS records are the most common records used in building a DNS server. These records are all in db files, which are database files used for each zone defined. For example, the zone ”.lab” will have a db.lab file. You can name it whatever, as long as it aligns with the named.conf.local zone path defined for the zone. It is best practice to have the domain

name as the db files extension. For both the A and NS records, their line begins with an @ symbol. Below is an example of my db.lab file for the lab zone on VM1.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.lab.

ns1     IN      A       192.168.107.129

jmu.lab.        IN      NS      ns1.jmu.lab.
ns1.jmu.lab.    IN      A       192.168.107.128
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

Figure 2.2: The db.lab file for the lab zone.

Start Of Authority Records

SOA records, or ”Start Of Authority” records, show the domain name that the specific db file has authority over. There are several sub records within. The MNAME is the domain name of the name server of the original or primary source of data for the zone. Like a .com, or for this study, .jmu and .lab. These are typically preceded with an ns1, which will come into play later with Namserver Records later. The next record is a RNAME, which is the Responsible Person record. For this project, I have root.jmu or root.lab for the root having responsibility. The next record is the serial number, which Bind uses to load different serial versions of the zone. This record has the space to be an unsigned 32 bit number. It should be updated every time there is a change to the db file. Some developers like to use the date it was edited on to help keep track of when it was last edited. Next, the refresh record is a 32 bit time interval before the zone should be refreshed. The next record is the retry record,

whish is another 32 bit time interval that would run out before a failed refresh should give up. Lastly, the expire record defines another 32 bit time value that gives an upper bound on the time interval that can run before the zone is no longer authoritative. Below is an example of an SOA record for BLANK zone.[3] An example cane be seen below in figure 2.3.

```
@       IN      SOA     ns1.lab. root.lab. (
                              5          ; Serial
                         604800          ; Refresh
                          86400          ; Retry
                        2419200          ; Expire
                         604800 )        ; Negative Cache TTL
```

Figure 2.3: An example of the SOA record in the parent servers db.lab, representing the start of authority for the zone.

Address Records

A records are address records, ”A” for short. These are the records you get when a domain maps to an IP address of a given domain. This is the most fundamental type of DNS record. There are also ”AAAA” records, which are for IPv6 traffic, but for this study we will only focus on IPv4 IPs. These records enable a user’s device to connect with and load a website, or whatever is at the IP address.[4] As seen below, in figure 2.4, when querying for an ip at one of my DNS Servers, I am returned with the IP address of that record.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab A +short
192.168.107.2
```

Figure 2.4: An example of digging for an A record, the short flag used to only give me the ip.

Below, figure 2.5, is an example of what these records look like.

```
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

Figure 2.5: An example of A records in the parent servers db.lab, computer.lab and bc.lab

Nameserver Records

NS records stand for ”Nameserver” records, and indicate which DNS server is authoritative for that domain. Like in the SOA, ns1.lab shows which server is authoritative for the .lab domain. Domains can have multiple NS records, like .lab has, which indicate primary and secondary nameservers for domains. For instance, in db.lab, there are multiple NS records, one pointing the the server itself and another pointing to the secondary server for the .jmu.lab domain.[5] See figure 2.6 for an example.

```
@          IN      NS      ns1.lab.

ns1        IN      A       192.168.107.129

jmu.lab.           IN      NS      ns1.jmu.lab.
ns1.jmu.lab.       IN      A       192.168.107.128
```

Figure 2.6: An example of NS records in the parent servers db.lab

2.2 The DNS Environment

There are numerous similarities between DNS with the internet and this testbed study. The bridge between the user and DNS servers is the DNS resolver or name resolution. In the project, the client digs at a server, and gets a response. This is a higher level abstraction, as there are no websites mapped to the IPs on my DNS servers, just more local IPs which could be a website. There is also no official ”.” or root domain, but my ”root” domain is lab., which is also my top level domain. This is seen in figure 2.7. In this setup, authority is held in the parent vm and passed downward onto the child server. In real DNS, a recursive resolver is used and traverses the different levels of domains.

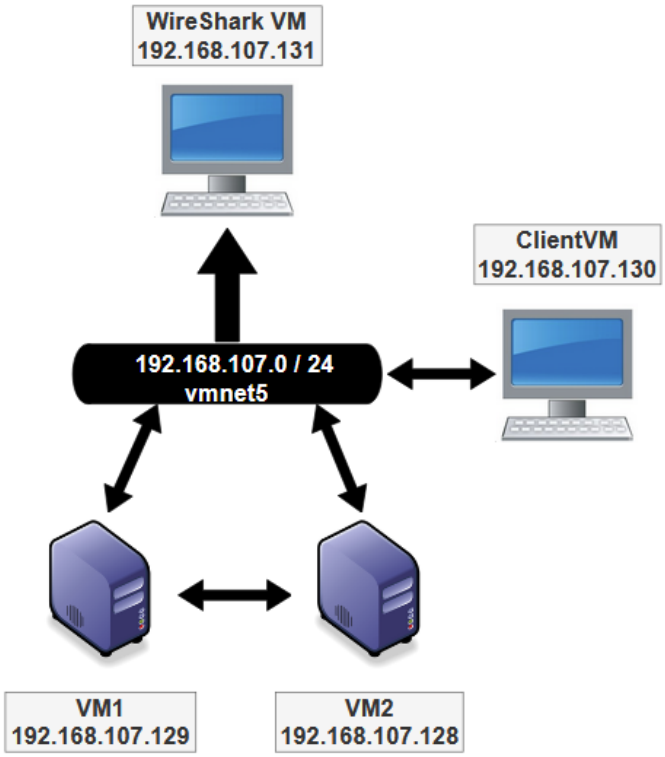

Figure 2.7: This is a diagram of the setup of the virtual environment, with inspiration of the layout coming from Dr.Aboutabl

2.2.1 Environment Zones

In the environment, there are different zones between the two name servers. As seen in figure 2.8, the parent zone holds .lab, with an extension to the child zone, .jmu. To be able to query the child, you need a domain name, followed by .jmu.lab. A full example of a valid query would be like this: quad.jmu.lab. Quad is the final piece, and this domain maps to 192.168.107.64. You can query the child server directly and receive responses, or you can query it through the parent server.

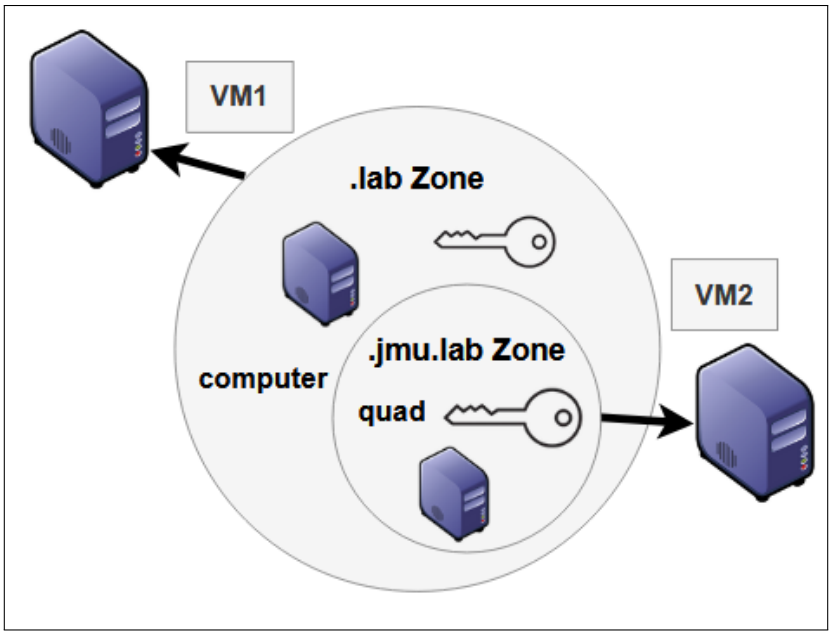

Figure 2.8: This is a diagram showing the zones and their relationship.

In this study, both the parent and the child server have authority over their own domains, and you can also traverse the tree to query the parent for information on the child server, as seen below in figure 2.9. There are also no IPV6 records, as this is disabled in the configuration of the servers.

```
;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.jmu.lab.

;; ADDITIONAL SECTION:
ns.jmu.lab.             86400   IN      A       192.168.107.128

;; Query time: 1 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Tue Feb 24 22:57:16 EST 2026
;; MSG SIZE  rcvd: 102

steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.128 quad.jmu.lab A +short
192.168.107.64
```

Figure 2.9: Querying the parent for information about the child, the querying the child with that information.

2.3 DNS Vulnerabilities

DNS without DNSSEC is very insecure and vulnerable to a plethora of attacks. Without DNSSEC, queries are readable to any person on the internet. An attacker could easily intercept this traffic and replace it with malicious content. This is where DNSSEC comes into play, helping to mitigate and prevent many attacks from occurring. It also proves that the information is coming from a trusted, authoritative source.

2.3.1 DNS Spoofing/Cache Poisoning

This is an especially vulnerable part of DNS without DNSSEC. This attack is a textbook man-in-the-middle attack, when the attacker intercepts a query and replaces it with their malicious information. DNS spoofing is when an attacker inserts their malicious address in place of a cached address. This attack is using DNS Hijacking in this instance, to redirect to what the attacker wants them to see. They pretend to be a DNS nameserver and forge a reply, and with UDP it makes it even easier for this attack to occur. Cache poisoning is DNS Spoofing without having to hijack the query. The systems involved with DNS, like servers, computers, and routers can cache DNS lookups for quicker recall. An attacker can poison the cache by replacing an entry with a spoofed domain. This spoofed domain gets accessed until the cache gets refreshed and the spoofed site gets replaced.[6]

According to the article, An Illustrated Guide to ..., there are a few criteria that need to be met in order for this attack to be successful. One is that the domain cannot be saved in the cache already, meaning that the attacker cannot directly modify the cache in this way and would have to wait for the Time To Live to expire. The attacker also has to guess the Query ID, which used to be easier to do, back when the Query ID was just incremented by one between queries. Now, the Query ID is random, which makes it much harder to guess. The attacker also has to guess the source port of the query, which is also random, making it even harder to guess. Query IDs also only used to be 16 bits long, which meant that brute

forcing 65,536 possible combinations of ids wasn't that difficult. Once this information is known by the attacker, they can forge a response much easier and avoid detection.

To actually perform this attack,

This can be seen in figure 2.10 below.

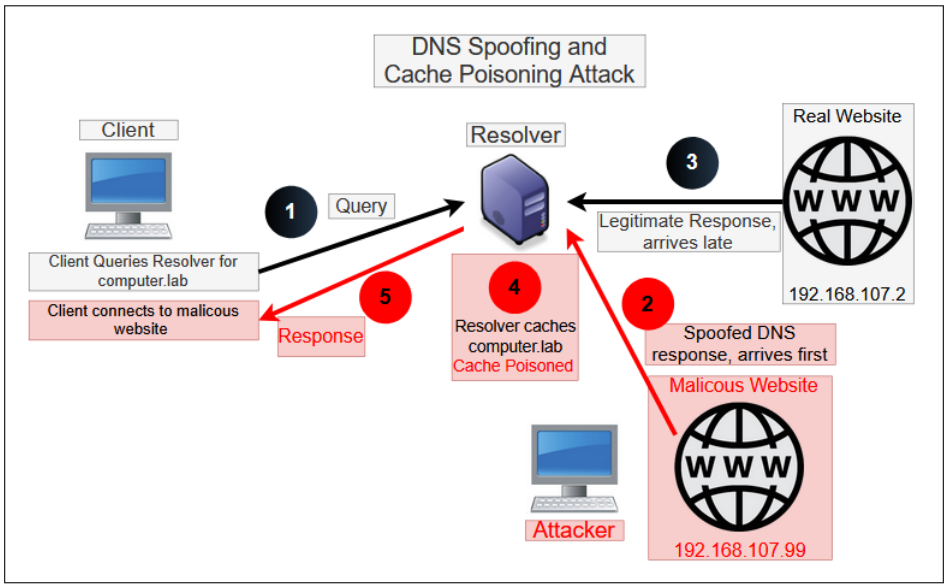

Figure 2.10: An example of DNS Spoofing, where the attacker intercepts a query and replaces it with their malicious information.

1. The client makes a query to the DNS server, asking for the IP address of a domain name.

2. The attacker intercepts this query and sends a forged response back to the client, containing the IP address of a malicious website instead of the legitimate one.

3. The DNS server receives the forged response and the legitimate information is never received.

4. The client receives the forged response and updates its cache with the malicious IP address. The client is now unknowingly directed to the attacker's website whenever it tries to access the legitimate domain name.

5. Finally, the attacker can use this access to the client's machine to steal sensitive information, such as login credentials or financial data, or to distribute malware.

With modern DNS implementations, there are various mitigations in place to prevent this attack from being successful. For example, randomizing the Query ID and source port makes it much harder for attackers to guess the necessary information to forge a response. Additionally,

2.3.2 DNS Hijacking

DNS Hijacking occurs when an attacker tries to redirect where the client is going with their query. The can perform a man-in-the-middle attack and take the query and replace it with their IP, causing the user's machine to directly communicate with the attacker. They could also give an IP that delivers a malicious payload. These are some of the most basic types of attacks when it comes the DNS hijacking[7]. In figure 2.11 below, it shows a diagram of how this attack works.

1. The attacker hijacks the DNS server, either by exploiting vulnerabilities in the server software or by gaining unauthorized access to the server. This allows the attacker to control the responses that the server sends back to clients, in this case changing the ip of computer.lab to the attacker's IP.

2. The client makes a query to the DNS server, asking for the IP address of a domain name.

3. The DNS server sends the malicious response back to the client, containing the IP address of a malicious website instead of the legitimate one.

4. Finally, the client unknowingly connects to the attacker's website, which can be used to steal sensitive information, such as login credentials or financial data, or to distribute malware.

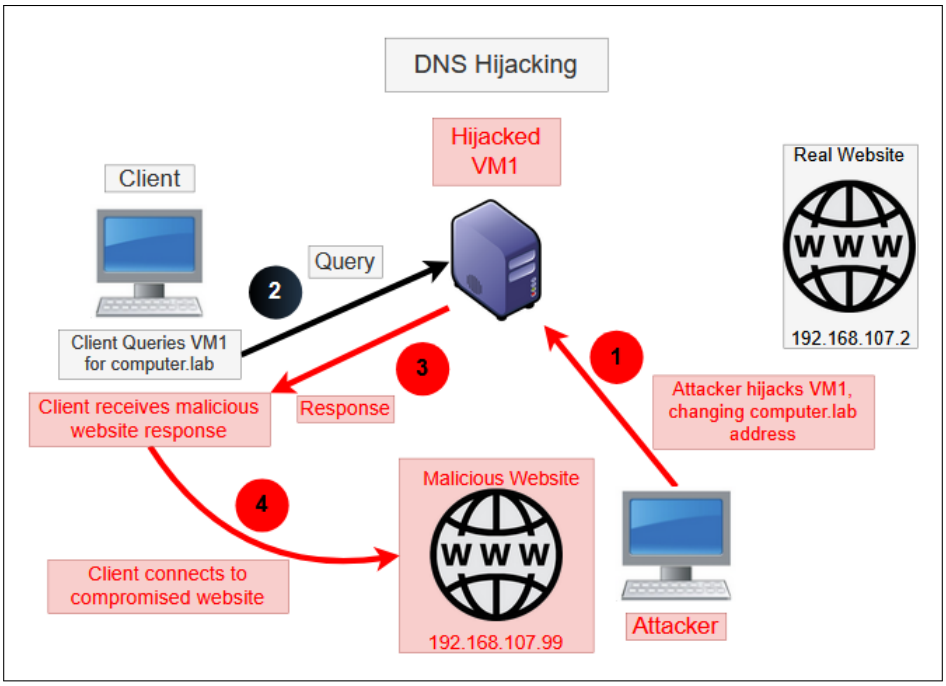

Figure 2.11: An example of DNS Hijacking, where the attacker redirects a query to a malicious server.

2.4 DNS Attacks

To show these attacks, I simulated them in the testbed environment. I assumed the role of the attacker and was able to hijack VM1, the root dns server of the virtual network. As seen in the figure below, the first step was to take control of the parent server and then change the db.lab file to have a malicious zone, which is shown in figure 2.12. Instead of being redirected to jmu.lab. NS, users are now redirected to the attacker's ns.evil.lab. This is a form of DNS Hijacking, and it also allows for DNS Spoofing and Cache Poisoning to occur. After hijacking the server, I was able to modify any record. Instead of just modifying the NS record, I could also modify an A record to have a malicious IP address, which would redirect users and resolvers to a different malicious IP address, making the attack complete and compromising the user.

After injecting this into the DNS server, I queried the parent server for the NS record of jmu.lab, and I was returned with the attacker's NS record, which is ns.evil.lab. This is

```
$TTL    86400
lab.    IN      SOA     ns.lab. root.lab. (
                             15         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu.lab.        IN      NS      ns.evil.lab.
ns.evil.lab.    IN      A       192.168.107.200

bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2

jmu.lab. IN DS 34939 8 2 9682051422A73107F882F98D04
```

Figure 2.12: After hijacking the parent server, I was able to change the db.lab file to have a malicious zone, which is shown above.

shown in figure 2.13. Without DNSSEC, there is no way for the client to know that this is not the real NS record, and they will be redirected to the attacker's server.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 jmu.lab. NS +norec

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 jmu.lab. NS +no
rec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52499
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 43a03a13c7ad7e9a0100000069b49377d30f4e3f16062eae (good)
;; QUESTION SECTION:
;jmu.lab.                       IN      NS

;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.evil.lab.
```

Figure 2.13: Successful injection of malicious DNS data.

In this query, the client is asking for the NS record of jmu.lab, and instead of getting the legitimate NS record, they are getting the attacker's NS record. This is a successful DNS Hijacking attack, and it also allows for DNS Spoofing and Cache Poisoning to occur.

Introduction to DNS Security Extension

3.1 DNSSEC Background

DNSSEC, or Domain Name System Security Extension, adds a layer of security to the otherwise vulnerable DNS. DNSSEC adds security by using cryptographic signatures with public key cryptography, with the private key signing it and the public key verifying. DNSSEC also implements hashing, which helps to add more confusion and making it harder for attackers to know what plain text strings were previously used. It also adds data integrity and and authentication to the data, showing whether it was tampered with and the signing provides the authentication.

There is an additional flag with dnssec queries, the ”ad” flag. This flag is passed along with successful DNSSEC queries, meaning ”Authenticated Data”. When you query for a domain with dnssec enabled, receiving this flag back instead of a SERVFAIL status means that the data was successfully authenticated.

This is NOT True? Should revisit. Do experiment where it doesn't validate. It validates now, still need to make a copy key and corrupt it.

You will not see it in the below screen shots, and that is because there isn't any validation going on yet. Seeing the records is enough for this part of my project, where as the next chapter I will write code to validate these records.

Below, figure 3.1, is db.lab, but now set for DNSSEC. It changed directories to allow for

permission to be more for the owner of the server (me). Originally, DNS configuration and the db files were in /etc/bind/, but now they moved to /var/lib/bind/ to allow for more freedom and not having to deal with bind ownership. Changing the zone configuration to point to the new db file allows the server to write into this directory. As you can see below, there is a good number of changes to the db file. Gone are the @ symbols preceding lines, and there is a new minimum time record. There is still a SOA and NS and A records, which have minimally changed from regular DNS. At the bottom of the figure, yuo can see a Delegation Signer record, which came from the child zone. This is an important part of the Chain of trust, allowing the parent to verify the child is the child.[8]

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F1
42C2
```

Figure 3.1: The new DNSSEC db.lab

3.2 New DNS Resource Records

With the regular DNS records previously discussed, there are some new records that must be taken into account. Regarding the Zone Signing Key and the Key Signing Key, the algorithms I used for the keys was RSASHA256, this is a good algorithm for encryption and one I am familiar with. The ZSK signs the records from the zone, and the KSK signs the ZSK. The hand shake looks a bit like this, an example of a query for computer.lab, figure 3.2.

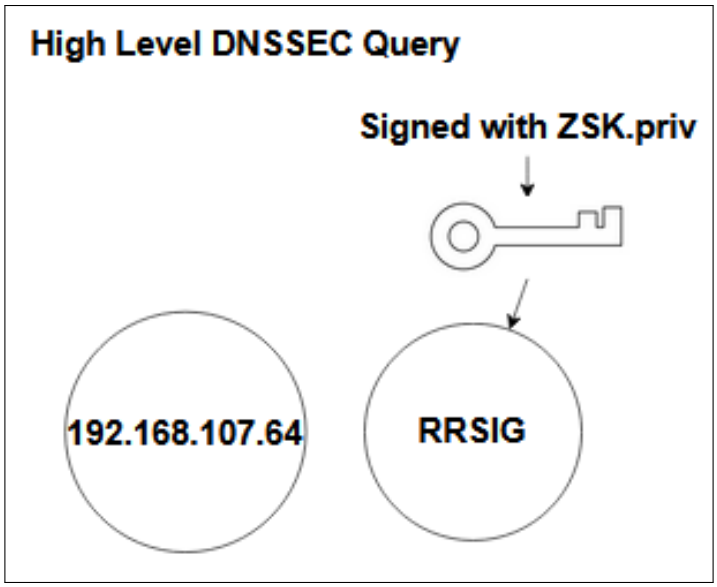

Figure 3.2: A diagram showing a high level view of a DNSSEC query, digging for an Address Record.

3.2.1 Resource Record Set

This record is not an explicit record, rather a descriptive word to describe records that are grouped together. For instance, if I query for my parent server for DNSKEYs, I receive the RRSet that is signed, seen below in figure 3.3. These keys are signed together as a set by the Key Signing Key, which the name is pretty explanatory for what it's purpose is. [9]

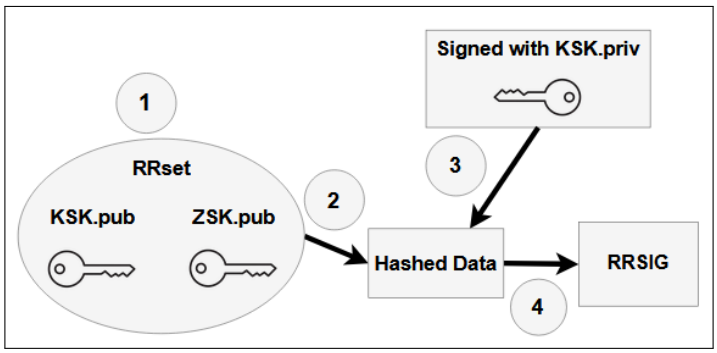

Figure 3.3: This query shows a dig at the parent for the DNSKEYs

Canonical Ordering

Talk about canonical ordering here. In some queries, there are multiple IP's that map to different IPs. [9]

Below is an example of

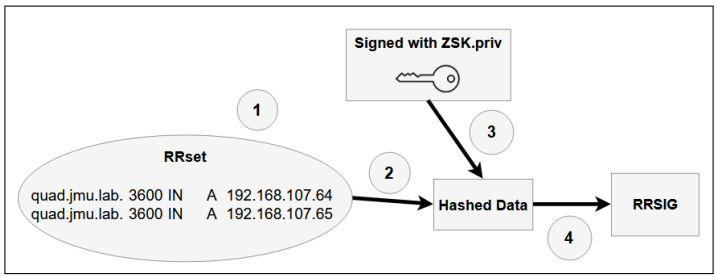

Figure 3.4: This query shows a dig at the parent for the DNSKEYs

3.2.2 Resource Record Signature

The Resource Record Signature, or RRSIG record, is used to verify the answers that are received. These are attached to every answer that is sent with DNSSEC. The signatures are used by recursive name servers to verify that the answer to the question is valid. [10] Below is an example of an address query with a signature, figure 3.5.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab +dnssec +multiline

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 computer.lab +dnssec +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19718
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 3df3cdc0dcb579af0100000069a5d721e88ca1e9eeb51c37 (good)
;; QUESTION SECTION:
;computer.lab.          IN A

;; ANSWER SECTION:
computer.lab.           86400 IN A 192.168.107.2
computer.lab.           86400 IN RRSIG A 8 2 86400 (
                                20260310233437 20260225023500 34104 lab.
                                GkosYckWQnxLuVJhdzymzDDRZuS+hBs5zzsdmrBCFkc3
                                4hdxuO8bbGjPQfkJTB9fXyov9xc6M1rDQko8+qzgD+9d
                                Kuextffp+oU3kiOLiJosuEUlDciTkVR/u4nm0T84STDN
                                RmK1yjPtp+hb4ZK0eLTIm3Ksqx/xQsT6Pl6Gl/Q= )

;; Query time: 0 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Mon Mar 02 13:29:53 EST 2026
;; MSG SIZE  rcvd: 248
```

Figure 3.5: Address record with a RRSIG from the ZSK

The is a lot of information in the fields that come with an RRSIG resource record. After the computer.lab domain name and the TTL, 86400, the first field is the type, being a RRSIG for an address record. After that, there is the algorithm field, 8 standing for RSASHA256. The next field is the labels field, which represents the count of labels in the domain name, excluding the root. The next field is the original TTL of the RRset, which is the value it is given in the authoritative zone. The field that follows the TTL is the signature expiration and inception fields. These are 32-bit unsigned integers, represented in the number of seconds since January 1st, 1970, or like the format above, being YYYYMMDDhhmmss, years, months, days, hours in 24 hour format, minutes and seconds. The latter format is distinguishable by having exactly 14 digits. The seconds option with be a 32-bit integer that won't be longer than 10 digits. The inception date specifies when it was created, and the expiration date being when it is no longer valid. Next is the key tag field, which holds the key tag of the signing key, in this instance the ZSK for the lab zone.

After the data that is sent with the RRSIG and is included in the hash. This is shown below in figure 3.6.

Field	Size	Description
Type Covered	2 bytes	RR type being signed (A=0x0001)
Algorithm	1 byte	8 = RSASHA256
Labels	1 byte	Number of labels in owner name
Original TTL	4 bytes	TTL from the RRSIG record
Sig Expiration	4 bytes	Unix timestamp, big endian
Sig Inception	4 bytes	Unix timestamp, big endian
Key Tag	2 bytes	Key tag of signing ZSK
Signer's Name	variable	Zone name in DNS wire format
Owner Name	variable	RR owner name in DNS wire format
Type	2 bytes	Same as type covered
Class	2 bytes	IN = 0x0001
TTL	4 bytes	Must match original TTL exactly
RDLENGTH	2 bytes	Length of RDATA
RDATA	variable	The actual record data (e.g. IP address)

Figure 3.6: A diagram showing the different fields of the RRSIG record, and what they represent.

All of the metadata inside the RRSIG Record, from the type to the keytag, gets concatenated and then hashed, then signed with the ZSK. This creates the signature which is the last piece of the RRSIG resource record puzzle.

3.2.3 DNSKEYs In General

DNSKEYs are the record that are used to sign an validate RRsets. These keys use public key cryptography, storing the public keys in DNSKEY RRs. These keys are used in different ways, and there are two different keys that we care about in this study, which are the Zone Signing Key and the Key Signing Key. The identifying type number that this RR is a DNSKEY is 48.[9]

The general format for these RRs is as follows, figure 3.7

```
;; QUESTION SECTION:
;lab.                    IN DNSKEY

;; ANSWER SECTION:
lab.                     3600 IN DNSKEY 257 3 8 (
                                AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqM
                                F19e5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4B
                                RP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF7oW
                                mcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZL
                                kalddzkc0KuvwAtmGsOTT7lkn3sx59JgWGSYiL+3FNZF
                                SZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr2
                                7TtP3LGJiBhcyMeJRT3i4bOBbCtwGelA6xq5i4k1fg2+
                                R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=
                                ) ; KSK; alg = RSASHA256 ; key id = 5852
lab.                     3600 IN DNSKEY 256 3 8 (
                                AwEAAbDODYwllXwQTD9OaDxw02d579560Yf4dGH7VpOb
                                +FcTTMIovHAAPuSgD0l5b1DnxWVhqupG9rM8b6FKxC+6
                                zNJycr9nUjTLkBxaVYgicGaS+X/0qy0K+d7spTg4qw3W
                                jzAls4LqquwljeMTNLe/peC2BZM/cmz0Mhn1MacYqqvH
                                ) ; ZSK; alg = RSASHA256 ; key id = 34104
lab.                     3600 IN RRSIG DNSKEY 8 1 3600 (
                                20260312172022 20260226164849 5852 lab.
                                sTOzJDy11sodWrqEb63MmZzOwhEfgO1ve7n/YgjL1ooQ
                                ECvhsNfYXM5D8Dq2R//oatu17DR9gT2fM591t/Hgi8lI
                                9B69/iPGOvArrKiK3PqmCRFjBnhl5T7ARcsvmA6Yedlj
                                yO2YitOs3aHYIZPoFhmRWrX0OwRA6rVO7/1JvdY/sZDI
                                Odi/oekocHkJ2pBdHP4tymB5nAiSn1Zm2RkGgNSJeQ1w
                                PvGccqXZIDuudTtI6Dy1+JTBZffvpYKwHupy3pCJJTdj
                                DNQ9rpsS15GligoH0fX4B1WmhvHt5CHFWm5/tDyrXYh
                                XIsiOP9XSx6X8RojnKATV+3DilfgbMgXnQ== )

;; Query time: 0 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
```

Figure 3.7: This query shows a dig at the parent for the DNSKEYs

These keys are stored in /var/cache/bind/, and are generated with the `dnssec-keygen` command. There are two parts of these keys, a ".key" and ".private", with the zone name and key tag in the name. There is an example of these stored records below. The keys are generated with the algorithm specified in the dnssec policy, which is RSASHA256 in this

case. Generally, the KSK is generated with a larger key size than the ZSK, because the KSK is used to sign the ZSK, which is used to sign the records. The KSK is more important to protect, because if it is compromised, then the ZSK can be compromised, and then all of the records can be compromised. The ZSK is used more frequently, so it is more likely to be compromised, but it is also easier to replace than the KSK.

Zone Signing Key

The Zone Signing Key, or ”ZSK” for short, is the key that is used to sign data from a zone. It signs all records except for RRset that are related to keys. The ZSK has the 256 flag, indicating that it is the ZSK. This key has a shorter Time To Live, or TTL, because it is sent out and used much more frequently than the Key Signing Key. Seen later under DNSSEC Policy, I set the time to live for the ZSK to be 60 days, where as the KSK is set to be a year. From what I saw, this is pretty standard.

Key Signing Key

The Key Signing Key, or ”KSK”, is used to sign what the ZSK does not sign: key-related RRsets. This keys serves another purpose, as it is the bridge between a parent and a child zone. The flag for the KSK is 257, indicating that this key is the KSK. Like the Zone Key, this key also follows the public key encryption scheme. This key pair is meant to protect the ZSK for the zone being queried.

3.2.4 Delegation Signer

The Delegation Signer Record, or DS record, is used to ”vouch” for another zone. The parent holds a copy of this in their zone, and when asked to query about the child zone, it can verify that the response comes from the child. This record is a hash that is stored of the child zone’s KSK, and the parent can use this hash to verify that the child zone is the child zone. This is an important part of the Chain of Trust, allowing the parent to verify the child

is the child, ensuring that the responses they receive can be trusted.[8]

These records are generated with the `dnssec-dsfromkey` command, which takes in the child zone's KSK public key and generates the DS record that can be stored in the parent zone. The output from this command gets pasted directly into the parents zone file, and then the parent can use this to verify the child zone's KSK. Below is an example of a DS record and the command used to generate it, figure 3.8. After adding this delegation signer record to the parent zone, I had to run an rndc reload the zone.

```
steve@steve-VMware-Virtual-Platform:~$ dnssec-dsfromkey -2 /var/cache/bind/Kjmu.
lab.+008+34939.key
jmu.lab. IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477A
C7F1342C2
```

```
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu.lab. IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F1342C2
```

Figure 3.8: The first image is the command used to generate the DS on the child server, then the output in the parent zone file.

3.3 Chain Of Trust

The Chain of Trust is how different levels of servers know they can trust each other. There must be trust through the whole process of the query, otherwise there is a vulnerable spot for intrusion by an attacker.The trust starts with a fitting name, the trust anchor.[11] An example of a trust anchor is below in figure 3.9. This record is stored on the client, and is a copy of the parent zone's KSK.

```
trust-anchors {
        lab. static-key 257 3 8 "AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqMF19e
5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4BRP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF
7oWmcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZLkalddzkc0KuvwAtmGsOTT7lkn3sx59JgWG
SYiL+3FNZFSZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr27TtP3LGJiBhcyMeJRT3i4bOBbCt
wGelA6xq5i4k1fg2+R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=";
};
```

Figure 3.9: An Example of my Trust Anchor

The trust anchor is the root of the trust, and the client can use this to verify the parent zone's KSK, which then can be used to verify the ZSK, which then can be used to verify the records

from the zone. This chain continues onto the child zone, where the parent can verify the child zone's KSK with the DS record, and then use that KSK to verify the ZSK, which then can be used to verify the records from the child zone. This is a repetitive process, where the parent can verify the child, and then the child can verify the next child, and so on. [12] A figure of the chain of trust is shown below in figure 3.11.

3.4 Key Rollovers

I ran into an issue when trying to run my validation code, where my ZSK had outlived it's 60 day TTL.

3.5 DNSSEC Options

Section to talk about my dnssec policy, key management, etc.

My client is the validator, with dnssec-validation set to yes. It has a trust anchor that was manually copied from the parent zone.

3.5.1 DNSSEC Policy

I adjusted the settings in the options configuration file, mainly to specify what algorithm to use, but also to set specifics for the keys. This is seen below in figure 3.10. Setting these settings also means that the keys are generated in the background with these specifications.

```
dnssec-policy rsa-policy {
        keys {
                ksk lifetime 365d algorithm RSASHA256 2048;
                zsk lifetime 60d algorithm RSASHA256 1024;
        };
};
```

Figure 3.10: This is what I added to my dnssec policy I created called "rsa-policy". This was then added to the zone configuration to implement this policy.

3.5.2 Signing

Right now, I am using Inline Signing, which automatically generates keys and signs the files. This setting also creates and keeps a separate signed zone file. Instead of running the `dnssec-signzone` and `dnssec-keygen` command, the server automatically signs the zone when it starts up. This is makes resigning the zone when it expires on the server easier, and also makes it easier to manage the keys. The server will automatically generate the keys for the zone, and then sign the zone with those keys. I just specified the algorithm, size, and time to live for the keys. It also reloads the zone automatically when the keys expire or zones are changed, which is a nice feature. This is a good option for this project, because it allows me to focus on the validation part of the project, rather than the signing part.

3.6 DNSSEC In The Environment

With DNSSEC on the internet, there is normally a validating resolver separate from the client machine that authenticates and validates the answers from the server before the client gets the response back. Normally, this is done recursively. For this study, there is no recursion and instead the answers will be manually validated.

Below is an example of the Chain of Trust in the environment figure 3.11.

3.7 Simulated Attacks with DNSSEC Enabled

Building off of the previous attacks covered in DNS Attacks, I will now talk about how these attacks are different with DNSSEC enabled. Firstly, I should mention that I had to use manual signing to simulate these attacks, because with inline signing, the server automatically resigns the zone when it is reloaded. This means that if I were to change the db file to have a malicious zone, I had to run the `dnssec-signzone` command to resign the zone with the new changes, and then reload the zone to have the changes take effect and

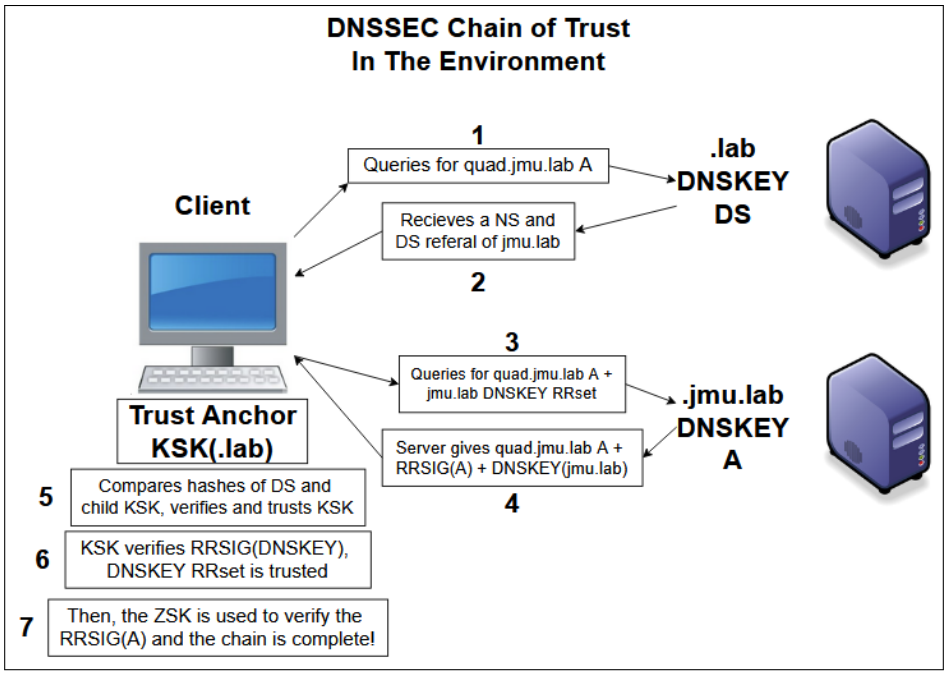

Figure 3.11: A diagram showing the Chain of Trust, with the trust anchor at the root, and the parent and child zones below. The arrows indicate the direction of trust, with each level trusting the next.

not get automatically signed. An example of this command is below in figure 3.12.

```
steve@steve-VMware-Virtual-Platform:/var/lib/bind$ sudo dnssec-signzone -K /var/
cache/bind -o lab. db.lab
Verifying the zone using the following algorithms:
- RSASHA256
Zone fully signed:
Algorithm: RSASHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                      ZSKs: 1 active, 0 stand-by, 0 revoked
db.lab.signed
```

Figure 3.12: A picture of the command used to manually sign the zone after making changes to the db file.

Below is an example of a valid `delv` command, which is used to validate the answers from the server. This command is used to show that the answers are valid and have not been tampered with. In this query, `delv` gets the address record for computer.lab and the corresponding RRSIG, and `delv` validates this in the background, showing that the answer is valid and has not been tampered with, as seen in figure 3.13.

After signing the data, I had to reload the zone to have the changes take effect. This is done with the `rndc reload` command, which updates the new configurations. So, to prove that

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A
; fully validated
computer.lab.           86400   IN      A       192.168.107.2
computer.lab.           86400   IN      RRSIG   A 8 2 86400 20260412223532 20260
313223532 34104 lab. NWPQcfaC2Th/jhCe5P/QihLYrXB9+CZJ8az8H6bft5L/xWRmTOgPdn26 2r
smskBIMSnGCeMUTgX+jnS4ET5Rp7FJbDdefkTi984fvUG5t0rccvE8 zTw5+NVfC2jw0kVP1ZgYRwRTM
eq8bklL6ImYI4b02kypF2wxYyD4wFXU JWA=
```

Figure 3.13: A picture of the command used to manually sign the zone after making changes to the db file.

DNSSEC is working, I signed the zone, then made a change to the db file to have a malicious zone, and then reloaded the zone. After doing this, I ran a dig query for computer.lab, and got the following response, figure 3.14.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 jmu.lab NS +rtrace
;; fetch: jmu.lab/NS
;; DNS format error from 192.168.107.129#53 resolving jmu.lab/NS for <unknown>:
non-improving referral
;; FORMERR resolving 'jmu.lab/NS/IN': 192.168.107.129#53
;; resolution failed: failure
```

Figure 3.14: This is the output of a `delv` command.

3.7.1 Corrupting DNS Data

To test validation with DNSSEC, I wanted to corrupt records, like the signatures and Trust Anchors to see what would happen. Below, in figure 3.15, is an example of deleting a single character from the RRSIG record, which is the signature that is used to validate the answer. As you can see, the RRSIG is not valid, because the data was tampered with, and the answer is not authenticated. This shows that DNSSEC is working, because it is able to detect that the data was tampered with and is not valid.

Testing just a corrupt signature was not enough, so I also wanted to test what would happen if the trust anchor was corrupted. Below, in figure 3.16, is an example of deleting a single character from the trust anchor, which is the root of the chain of trust. With a corrupted trust anchor, the client cannot verify the parent zone's KSK, which means that the client cannot verify any of the records from the parent zone or any of the child zones.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A +rtrace
;; fetch: computer.lab/A
;; fetch: lab/DNSKEY
;; validating computer.lab/A: no valid signature found
;; RRSIG failed to verify resolving 'computer.lab/A/IN': 192.168.107.129#53
;; resolution failed: RRSIG failed to verify
```

Figure 3.15: The output above shows a `delv` command for computer.lab, but the RRSIG is not valid, because the data was tampered with.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A
;; validating lab/DNSKEY: no valid signature found (DS)
;; no valid RRSIG resolving 'lab/DNSKEY/IN': 192.168.107.129#53
;; broken trust chain resolving 'computer.lab/A/IN': 192.168.107.129#53
;; resolution failed: broken trust chain
```

Figure 3.16

Algorithms and Data Structures For DNSSEC

4.1 An Introduction to DNSSEC Signing

Earlier, different records were explained and new resource records were introduced, but how do they actually get signed? Before signing can occur, keys must be generated with `dnssec-keygen`. This tool creates the necessary public and private keys that will be used for signing the zone file. After the keys are generated, the signing process can be manually completed using a tool called `dnssec-signzone`. Running this tool in the command line will create the necessary signatures for the zone file.

The data is built according to how it is laid out previously in Resource Record Signature. In my implementation, I read in a file with the hexadecimal representation of the data, pulled from Wireshark captures.

4.1.1 Hashing

The algorithm of choosing for hashing in this project is SHA-256. This algorithm is widely used in many cryptographic applications, and is considered to be a secure hashing algorithm. It produces a 256-bit hash, which is a fixed size output that is unique to the input data. This means that even a small change in the input data will produce a completely different hash, making it difficult for attackers to create a hash collision. Having 2^{256} possible combinations of hashes makes it infeasible for attackers to brute force their way to a collision, which is

when two different inputs produce the same hash. With this collision resistance, SHA-256 is a good choice for hashing in DNSSEC, as it helps to ensure the integrity of the data being signed and verified.

Possible example of how different two hashes are?

4.1.2 Signing

Rather than having recursion validate the signatures, I am writing C code to do this instead. From using OpenSSL to creating structs to store data, there were various algorithms and data structures that were used to accomplish this task.

In order to validate the signatures, I had to implement various algorithms for hashing and signing the data. For example, I had to implement the RSA algorithm for signing the data, and the SHA-256 algorithm for hashing the data.

In the environment, I am manually signing these records myself, and then using the public key to verify the signatures. This is a good way to test the validation process, because I can control the data that is being signed and verified. Not only that, but I am also creating the signature myself to prove that the received signature is valid as well.

4.1.3 Key Conversion

In order to use the keys generated in my lab environment, I had to convert them from the format that they were generated in to a format that could be used in my C code. The keys were generated in the DNSSEC format, which had extra information and wasn't formatted correctly, so I needed to convert them to PEM format to use in my code.

4.2 RRSIG Construction

To build the RRSIG records, I followed the format laid out in Resource Record Signature. I read in the data from a file, which was pulled from Wireshark captures, and then built the

RRSIG record according to the format. First, the RRSIG metadata is built, which includes the type of record, the algorithm used, the labels, the original TTL, the signature expiration and inception times, the key tag, and the signer's name. Then, the canonicalized RRSet is concatenated to the data, and the whole thing is hashed using SHA-256. Finally, the hash is signed using the RSA algorithm with the private key, and the signature is added to the RRSIG record.

4.3 OpenSSL

For this project, we used OpenSSL behind the scenes with the tools we ran in the previous section. OpenSSL is a powerful library that provides various cryptographic functions, including those needed for DNSSEC signing. It allows us to generate keys, create signatures, and manage certificates, which are essential components of the DNSSEC infrastructure. OpenSSL even provides support for converting data from Base64 into binary

When we run the `dnssec-signzone` command, it utilizes OpenSSL to perform the cryptographic operations required for signing the zone file. By doing this, it hashes the data and then signs it with the private key in the background.

To complete the signing process, OpenSSL provides a function called EVP_DigestSign, which allows us to create a signature using the specified hashing algorithm and private key. This function takes care of the details of the signing process, making it easier for us to implement DNSSEC signing in our code. Rather than having to separately hash and then sign the data, this function takes care of both.

Programming the Signature and Validation of Several DNSSEC Resource Records

5.1 Main Functionality

There are several arguments that can be passed to the program:

1. The first argument is the name of the file that contains the hexadecimal representation of the data to be signed.

2. The second argument is the name of the file that will contain the signature from the server in Base64 format.

3. The third argument is the name of the file that contains the private key that will be used for signing.

4. The final argument is the name of the file that contains the public key that will be used for verifying the signature.

Using these arguments, the data is read into buffers in the program, and the base64 signature is read in, converted to binary, and stored in a buffer as well.

5.2 Converting Base64 Data

To convert the Base64 signature from the file into a format that can be used for verification, I used OpenSSL's Base64 decoding functions. This allows me to take the Base64-encoded signature and convert it back into its original binary format, which is necessary for the verification process. The code for this conversion is shown in **??**. I enlisted Anthropic's Claude Code Interpreter to help me with this part, and it was able to generate the code for me based on the OpenSSL documentation. This allowed me to solely focus on the actual signing and validation portions of the project.

5.3 Hashing the Data

To hash the data, I used OPENSSL's SHA256 algorithm. This algorithm is widely used in many cryptographic applications, and is considered to be a secure hashing algorithm. It produces a 256-bit hash, which is a fixed size output that is unique to the input data. This means that even a small change in the input data will produce a completely different hash, making it difficult for attackers to create a hash collision. The code that just hashes the data is shown in figure 5.1.

```
unsigned char digest[32];
EVP_MD_CTX *ctx = EVP_MD_CTX_new ();
EVP_DigestInit_ex (  ctx , EVP_sha256 () , NULL  );
EVP_DigestUpdate (  ctx , data , len  );
unsigned int digest_len;
EVP_DigestFinal_ex (  ctx , digest , &digest_len  );
EVP_MD_CTX_free (  ctx  );
```

Figure 5.1: Code from a helper function that hashes the data using OpenSSL's SHA256 algorithm, used for printing the hash of the data to be signed.

5.4 Signing the Data

For signing the hashed data, RSA keys were used. For signing, both public and private keys were needed. The private key is used to sign the data, and the public key is used to verify the signature. The verification code is shown in Verifying the Signature. The code that just signs the data is shown in figure 5.2.

```
// --- Generate the signature ---
unsigned char *gen_sig = NULL;
size_t gen_sig_len = 0;

EVP_MD_CTX *sign_ctx = EVP_MD_CTX_new();
if (EVP_DigestSignInit(sign_ctx, NULL, EVP_sha256(), NULL, privkey) != 1 ||
    EVP_DigestSignUpdate(sign_ctx, sigbase, sigbase_len) != 1 ||
    EVP_DigestSignFinal(sign_ctx, NULL, &gen_sig_len) != 1) {
    fprintf(stderr, "Error: signing init failed\n");
    return 1;
}

gen_sig = malloc(gen_sig_len);

if (EVP_DigestSignFinal(sign_ctx, gen_sig, &gen_sig_len) != 1) {
    fprintf(stderr, "Error: signing final failed\n");
    return 1;
}
```

Figure 5.2: This code is showing how the signature is created using OpenSSL's RSA signing functions. I am assuming the role of the DNS Server in this code, using the private key to sign the data.

5.5 Verifying the Signature

The code in figure 5.3 shows how the signature is verified using OpenSSL's RSA verification functions. I am assuming the role of the DNS Client in this code, using the public key to verify the signatures. First, I verify the generated signature, which is the signature that I created in the previous section. Then, I verify the received signature, which is the signature that was read in from the file that was converted from Base64. If both of these return without error, both signature are then passed to the helper function compare_sigs.

```c
// --- Verify our signature with public key ---
printf("=== Verify: generated signature with public key ===\n");
EVP_MD_CTX *verify_ctx = EVP_MD_CTX_new();
if (EVP_DigestVerifyInit(verify_ctx, NULL, EVP_sha256(), NULL, pubkey) == 1 &&
    EVP_DigestVerifyUpdate(verify_ctx, sigbase, sigbase_len) == 1 &&
    EVP_DigestVerifyFinal(verify_ctx, gen_sig, gen_sig_len) == 1)
    printf("PASS: generated signature verifies correctly\n\n");
else
    printf("FAIL: generated signature did not verify\n\n");
EVP_MD_CTX_free(verify_ctx);

// --- Verify existing RRSIG with public key ---
printf("=== Verify: existing RRSIG with public key ===\n");
verify_ctx = EVP_MD_CTX_new();
if (EVP_DigestVerifyInit(verify_ctx, NULL, EVP_sha256(), NULL, pubkey) == 1 &&
    EVP_DigestVerifyUpdate(verify_ctx, sigbase, sigbase_len) == 1 &&
    EVP_DigestVerifyFinal(verify_ctx, existing_sig, existing_sig_len) == 1)
    printf("PASS: existing RRSIG verifies correctly\n\n");
else
    printf("FAIL: existing RRSIG did not verify - sigbase may not match exactly\n\n");
EVP_MD_CTX_free(verify_ctx);

// --- Binary comparison ---
compare_sigs(gen_sig, gen_sig_len, existing_sig, existing_sig_len);
```

Figure 5.3: Code for verifying the signature using OpenSSL's RSA verification functions.

Additionally, the helper function at the bottom of that code snippet is used to compare the binaries of each signature.

5.6 Generating and Validating an A RRSIG Record

5.7 Generating and Validating an NS RRSIG Record

5.8 Generating and Validating an DNSKEY RRSIG Record

```
=== Existing RRSIG signature (128 bytes) ===
0000  63 bf 85 25 2a 93 a8 35  2b 1f 05 21 e8 fb 67 40  |c..%*..5+..!..g@|
0010  35 28 1f ec 8b 6c dc cd  69 aa 6c 76 70 20 88 d3  |5(...l..i.lvp ..|
0020  19 f7 a6 91 28 5f fe b7  29 72 36 a3 0a 45 03 9d  |....(_..)r6..E..|
0030  06 13 a1 74 2c 5b 69 6d  c1 ab a0 45 e3 7f 56 c6  |...t,[im...E..V.|
0040  83 ec dc b2 04 af 04 dd  37 6e 99 5d ec 72 22 2b  |........7n.].r"+|
0050  99 1d ac 3d 38 b8 0f 02  b5 82 c1 a5 db 85 c7 3b  |...=8..........;|
0060  46 2f 9a e7 cb 83 ba b2  d0 fd c9 4e 66 d2 97 13  |F/.........Nf...|
0070  56 d4 90 c7 13 f4 20 2c  77 13 cd 2e 39 d8 e1 2a  |V..... ,w...9..*|


=== Our generated signature (128 bytes) ===
0000  63 bf 85 25 2a 93 a8 35  2b 1f 05 21 e8 fb 67 40  |c..%*..5+..!..g@|
0010  35 28 1f ec 8b 6c dc cd  69 aa 6c 76 70 20 88 d3  |5(...l..i.lvp ..|
0020  19 f7 a6 91 28 5f fe b7  29 72 36 a3 0a 45 03 9d  |....(_..)r6..E..|
0030  06 13 a1 74 2c 5b 69 6d  c1 ab a0 45 e3 7f 56 c6  |...t,[im...E..V.|
0040  83 ec dc b2 04 af 04 dd  37 6e 99 5d ec 72 22 2b  |........7n.].r"+|
0050  99 1d ac 3d 38 b8 0f 02  b5 82 c1 a5 db 85 c7 3b  |...=8..........;|
0060  46 2f 9a e7 cb 83 ba b2  d0 fd c9 4e 66 d2 97 13  |F/.........Nf...|
0070  56 d4 90 c7 13 f4 20 2c  77 13 cd 2e 39 d8 e1 2a  |V..... ,w...9..*|

=== Verify: our signature with public key ===
PASS: our signature verifies correctly

=== Verify: existing RRSIG with public key ===
PASS: existing RRSIG verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
```

Figure 5.4: Output from validating an A RRSIG record.

```
=== Existing RRSIG signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a..........].|
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9Zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|


=== Our generated signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a..........].|
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9Zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|

=== Verify: our signature with public key ===
PASS: our signature verifies correctly

=== Verify: existing RRSIG with public key ===
PASS: existing RRSIG verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
```

Figure 5.5: Output from validating an NS RRSIG record.

```
=== Existing RRSIG signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a..........].|
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9Zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|


=== Our generated signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a..........].|
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9Zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|

=== Verify: our signature with public key ===
PASS: our signature verifies correctly

=== Verify: existing RRSIG with public key ===
PASS: existing RRSIG verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
```

Figure 5.6: Output from validating an DNSKEY RRSIG record.

Bibliography

[1] BIND9, "Introduction to dns and bind 9." https://bind9.readthedocs.io/en/v9.18.39/chapter1.html#the-domain-name-system-dns, 2025. Accessed: 2026-1-1.

[2] Cloudflare Learning, "What is dns?." https://www.cloudflare.com/learning/dns/what-is-dns/, 2025. Accessed: 2025-12-22.

[3] P. Mockapetris, "Domain names - implementation and specification." https://www.rfc-editor.org/rfc/rfc1035, 1987. Accessed: 2026-1-1.

[4] Cloudflare Learning, "Dns a record." https://www.cloudflare.com/learning/dns/dns-records/dns-a-record/, 2025. Accessed: 2026-1-20.

[5] Cloudflare Learning, "Dns ns record." https://www.cloudflare.com/learning/dns/dns-records/dns-ns-record/, 2025. Accessed: 2026-1-20.

[6] Cloudflare Learning, "What is dns cache poisoning? — dns spoofing." https://www.cloudflare.com/learning/dns/dns-cache-poisoning/, 2025. Accessed: 2026-2-22.

[7] Imperva, "What is dns hijacking / redirection attack." https://www.imperva.com/learn/application-security/dns-hijacking-redirection/, 2026. Accessed: 2026-2-26.

[8] Ubuntu Server, "Installing dns security extensions." https://documentation.ubuntu.com/server/how-to/networking/install-dnssec/, 2026. Accessed: 2026-2-20.

[9] R. Arends, et. al, "Resource records for the dns security extensions." https://datatracker.ietf.org/doc/html/rfc4034, 2005. Accessed: 2026-2-23.

[10] BIND9, "Dnssec guide." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html, 2025. Accessed: 2026-2-25.

[11] BIND9, "Trust anchor." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html#trust-anchors, 2025. Accessed: 2026-2-25.

[12] dnsimple, "The dnssec chain of trust." https://support.dnsimple.com/articles/dnssec-chain-of-trust/, 2026. Accessed: 2026-2-28.

Appendix

A.1 Procedure Manual